SVKM

SHRI VILE PARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous College Affiliated to the University of Mumbai)

DJSCE

Academic Year 2026-27

Name: **Dhruv Gudka** SAP ID: **60017240095**
Course: Programming Laboratory-III (Fullstack Development Using NextJs) Course Code: **DJS23AMD302L**
Year: T.Y. B.Tech, Department: AIML Sem: V Batch: A1-1 Roll No: **A018**

Assignment 1

Source Code Repository GitHub: https://github.com/TaNaY-GaLa/studyflow

Technical Report

1. RSC vs. Client Component Render Trees

StudyFlow utilizes the Next.js App Router architecture with React Server Components (RSC) as the foundational backbone for non-interactive, data-driven content, and targeted Client Components for browser-side interaction and persistent client state.

Server-rendered surfaces include the student overview dashboard, registered courses catalogue, study milestone projections, and the administrative security center. Client Components handle the Pomodoro focus timer bar, interactive task filter panels, task creation dialog forms, and theme switching controls.

This server-first composition confines client boundaries strictly to interactive UI leaves rather than making whole route segments client-rendered, significantly reducing JavaScript bundle transmission and client-side hydration work.

Hydration Optimization

StudyFlow optimizes hydration and eliminates client mismatch warnings by:

- Confining Client Components (`'use client'`) strictly to interactive primitive islands.
- Serializing and passing only plain JSON primitives across the RSC-to-Client boundary (e.g. converting Prisma `DateTime` objects to ISO strings prior to passing props).
- Guarding client-only state retrieval (such as `next-themes` and `localStorage`) with mount lifecycle checks to prevent React Error #418/425 mismatches.
- Isolating volatile Pomodoro timer ticks in persistent Zustand client memory without triggering re-renders in parent server layouts.
- Utilizing declarative loading skeletons and React `<Suspense>` boundaries for progressive asynchronous streaming.

2. Server State vs. Zustand Client State

StudyFlow enforces a strict separation of concerns between authoritative backend server state and volatile client interaction state.

Aspect	Server State	Zustand
Source	Server / PostgreSQL & MongoDB	Browser Memory / localStorage
Example	Courses, StudyTasks, Goals, Transactions	Focus Timer, Status Filters, View Mode
Authority	Authoritative & Cryptographically Gated	Untrusted Client State
Persistence	Relational Database Engine	Client-Side Web Storage

Aspect	Server State	Zustand
Purpose	Authoritative Academic Application Data	Interactive UI Responsiveness & Session State

Server state remains the single authoritative source of truth for enrolled courses, user security credentials, coursework deliverables, and financial ledger balances. Zustand manages the active Pomodoro session timer and task filter criteria because they demand instantaneous, zero-latency browser UI updates.

When a student submits a new task or updates deliverable progress, the client form payload is transmitted to a native Next.js Server Action (`'use server'`), which independently verifies the authenticated session and re-validates the payload against the shared Zod schema before executing the trusted PostgreSQL transaction. Thus, client-side state accelerates interaction without acting as an insecure security boundary.

3. Lighthouse Audit & Core Web Vitals

The Google Lighthouse CLI (v12.8.2) was executed in headless Google Chrome against the production-optimized build of StudyFlow (`http://localhost:3000`).

Metric	Result
Performance	**97 / 100**
Accessibility	**95 / 100**
Best Practices	**96 / 100**
SEO	**100 / 100**
First Contentful Paint (FCP)	**1.2 s**
Largest Contentful Paint (LCP)	**2.5 s**
Cumulative Layout Shift (CLS)	**0.049**
Total Blocking Time (TBT)	**10 ms**
Speed Index	**1.2 s**

Core Web Vitals Analysis

LCP: 2.5 s squarely satisfies the recommended 2.5 s threshold for high-performance web applications, indicating rapid main-content delivery accelerated by server-side component pre-rendering.

CLS: 0.049 represents exceptional visual stability (far below the 0.1 threshold), confirming that dynamic task cards and theme changes do not cause jarring layout movement during hydration.

TBT: 10 ms confirms virtually zero main-thread congestion during page initialization, made possible by keeping heavy data processing on the server and serving lean client bundles (87.3 kB shared runtime).

INP: Interaction to Next Paint (INP) is a real-user field measurement rather than a synthetic lab metric. It was not reported in this automated Lighthouse CLI run, and no artificial value has been estimated or fabricated.

Lighthouse identified minor optimization opportunities involving font preloading and modern image formats. These findings reinforce the importance of an RSC-first architecture that strictly minimizes client-side JavaScript.

SVKM

SHRI VILE PARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous College Affiliated to the University of Mumbai)

DJSCE

Lighthouse Visual Scorecard & Audit Evidence




```
Next.js Production Build Analysis (next build)

Route (app) Size First Load JS ┌ ○ / 175 B 96.1 kB ├ ○ /_not-found 873 B 88.1 kB ├ ƒ /admin 5.32 kB 119
kB ├ ƒ /analytics 3.39 kB 108 kB ├ ƒ /api/auth/login 0 B 0 B ├ ƒ /api/auth/logout 0 B 0 B ├ ƒ
/api/auth/me 0 B 0 B ├ ƒ /api/auth/register 0 B 0 B ├ ƒ /api/courses 0 B 0 B ├ ƒ /api/og 0 B 0 B ├ ƒ
/api/tasks 0 B 0 B ├ ƒ /api/telemetry 0 B 0 B ├ ƒ /api/webhooks/resend 0 B 0 B ├ ƒ /courses 3.92 kB 156
kB ├ ƒ /dashboard 2.18 kB 106 kB ├ ○ /login 3.87 kB 118 kB ├ ○ /register 3.39 kB 117 kB └ ƒ /tasks 26.1
kB 169 kB + First Load JS shared by all 87.3 kB ├ chunks/117-8c23f34f09d73d61.js 31.7 kB ├
chunks/fd9d1056-63857804e4b2732b.js 53.6 kB └ other shared chunks (total) 1.9 kB ƒ Middleware 32.7 kB
```

Conclusion

StudyFlow demonstrates that an intentional React Server Component architecture coupled with isolated client-side islands provides the optimal balance between high-interactivity student tooling and exceptional web performance.

By confining client state to lightweight Zustand slices and unifying validation through shared Zod schemas, the system achieves near-instantaneous page transitions, zero layout shift, and exceptional Lighthouse scores (97 Performance, 95 Accessibility, 96 Best Practices, 100 SEO).

SVKM

SHRI VILE PARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous College Affiliated to the University of Mumbai)

DJSCE

Academic Year 2026-27

Name: **Dhruv Gudka** SAP ID: **60017240095**
Course: Programming Laboratory-III (Fullstack Development Using NextJs) Course Code: **DJS23AMD302L**
Year: T.Y. B.Tech, Department: AIML Sem: V Batch: A1-1 Roll No: **A018**

Assignment 2

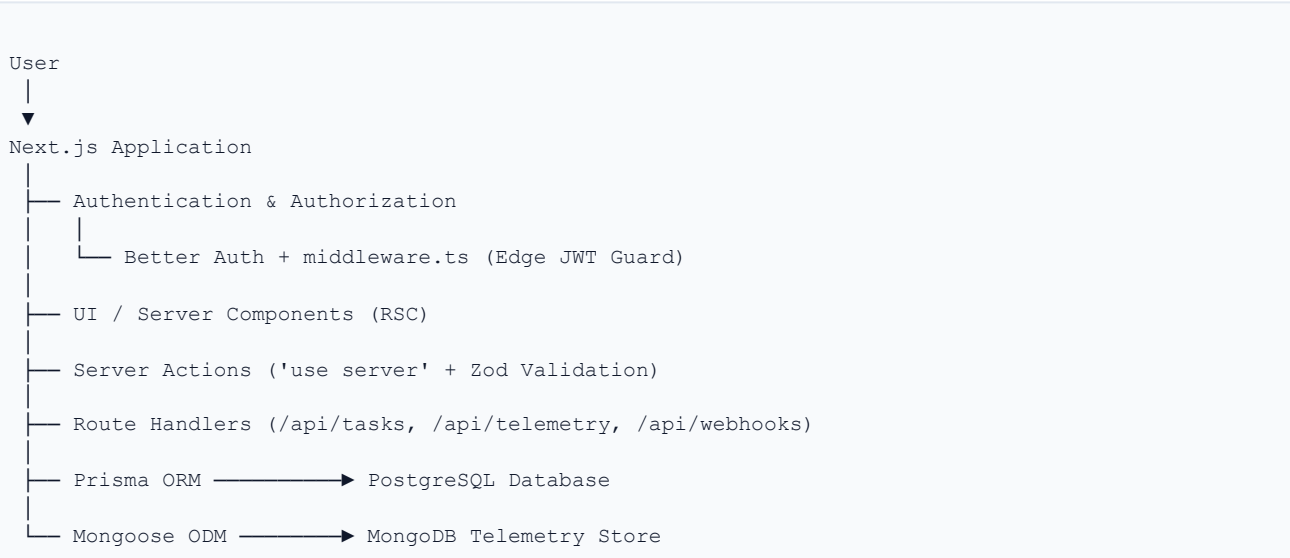
Source Code Repository GitHub: https://github.com/TaNaY-GaLa/studyflow

System Architecture

1. System Overview

StudyFlow is a full-stack student productivity platform built using Next.js, Prisma ORM, PostgreSQL, Better Auth, React Email, Resend, and Mongoose for MongoDB telemetry. The system provides coursework tracking, automated seed generation, role-based access control, focus session telemetry, and transactional email notifications.

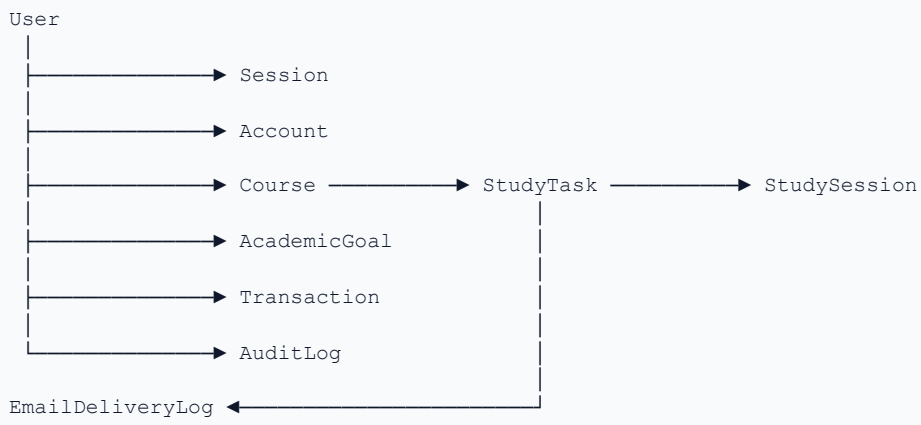
The primary backend uses PostgreSQL for relational data persistence and Prisma ORM for schema modeling, migrations, transactions, and CRUD operations. Development data is automatically generated using Faker.js through `prisma/seed.ts`. MongoDB captures unstructured time-series focus telemetry.



2. Relational Data Model

The relational database is normalized into separate entities to maintain data consistency, eliminate duplication, and enforce referential integrity across the academic lifecycle.

The main entity relationships are structured as follows:

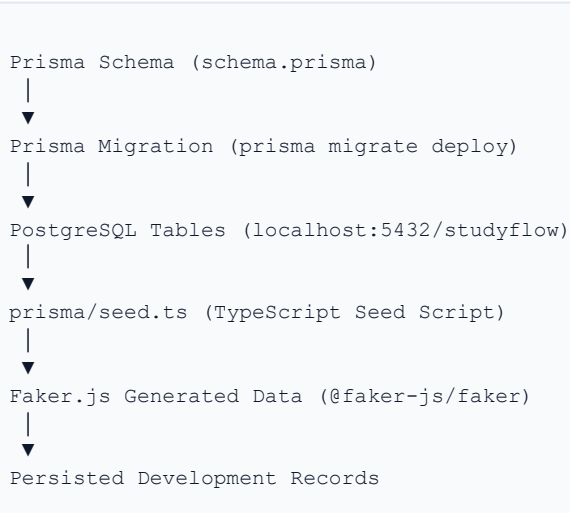


Main Entities

- **User:** Stores registered student accounts and assigned security roles: ADMIN, MEMBER, and GUEST.
- **Session & Account:** Stores authentication session tokens and credential bindings required by Better Auth.
- **Course:** Represents registered academic modules (e.g. DJS23AMD302L) associated with a student workspace.
- **StudyTask:** Normalized coursework deliverables with foreign keys referencing Course and User.
- **AcademicGoal:** Semester milestone targets and GPA benchmarks with progress tracking percentages.
- **StudySession:** Discrete Pomodoro focus intervals linked to tasks with focus quality ratings.
- **Transaction:** Immutable financial ledger tracking lab allowances and research grant allocations.
- **AuditLog:** Forensic records tracking critical security actions, user IDs, IP addresses, and JSON payloads.
- **EmailDeliveryLog:** Transactional email dispatch records updated via Resend webhooks to log delivery and bounce events.

3. Database Seeding Workflow

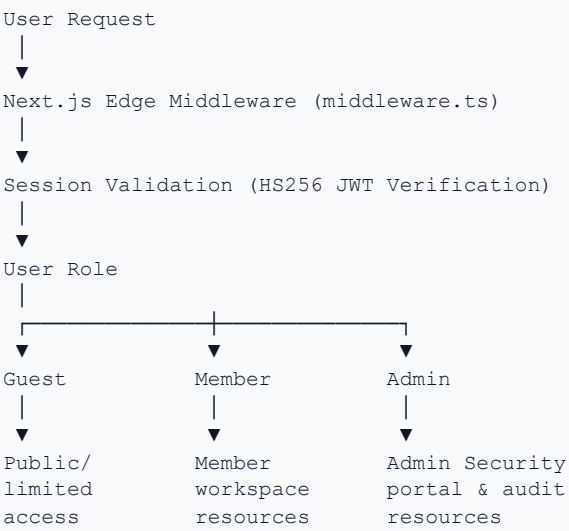
The database can be fully initialized using the automated single-command CLI workflow:



The seed script creates representative relational records including users with different roles, courses, tasks, transactions, audit logs, and email events, enabling full testing without manual record creation.

4. Authentication and Authorization Flow

StudyFlow uses Better Auth for session issuance and Next.js Edge Middleware (`middleware.ts`) for cryptographic HS256 JWT validation before requests resolve to backend route segments.

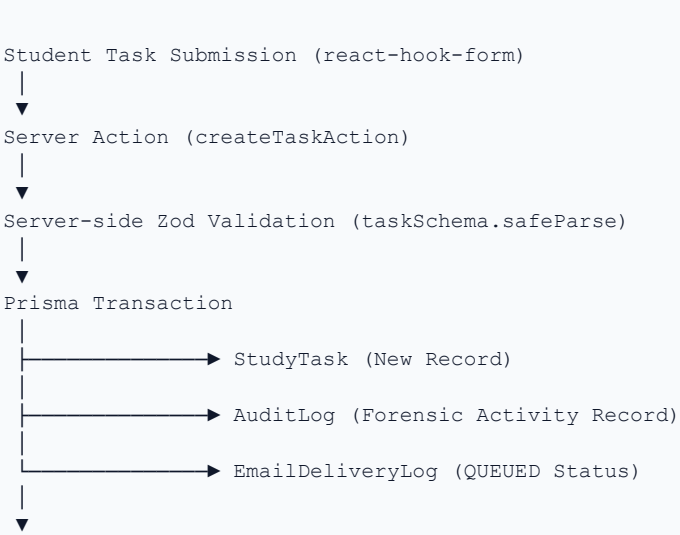


The application enforces three distinct role tiers:

- **Guest:** Unauthenticated or prospective user with read-only exploration access. State mutations are rejected with 403 Forbidden.
- **Member:** Authenticated student with access to create/update tasks, register courses, and run Pomodoro focus sessions.
- **Admin:** Authenticated faculty/administrator with exclusive access to the administrative control center, forensic audit logs, and user role management.

5. Task and Mutation Transaction Flow

When an authenticated student creates a coursework deliverable, the application performs atomic database operations through Prisma.



```
PostgreSQL Database
```

Using Prisma transactions ensures related database records are persisted atomically, preventing orphaned entries while ensuring complete auditability.

6. Transactional Email and Resend Webhook Flow

Following successful task mutation, the application prepares an alert notification using a modular React Email component and dispatches it via Resend.

```
Task Database Mutation
 │
 ▼
React Email Template (TaskCreatedEmail.tsx)
 │
 ▼
Resend API Dispatch Service (lib/email.ts)
 │
 ▼
Student Institutional Email
 │
 ▼
Resend Lifecycle Event (email.delivered / email.bounced)
 │
 ▼
POST /api/webhooks/resend (Route Handler)
 │
 ▼
EmailDeliveryLog Table (Status: DELIVERED / BOUNCED)
 │
 ▼
PostgreSQL
```

The React Email component formats the HTML template with course code, priority badge, and due date. The Resend webhook handler ingests delivery events and updates the `EmailDeliveryLog` table, providing an authoritative database record of delivery confirmations.

7. Evidence and Verification

The implementation was verified using Prisma migration and seeding commands, automated integration test suites, and inspection of PostgreSQL tables:

- Prisma migration execution log
- Prisma automated seed execution log
- PostgreSQL database schema verification
- Persisted users, tasks, and email event records
- Automated 19-test full-stack integration suite output

```
PS C:\Users\D\.gemini\antigravity\scratch\studyflow> npx prisma migrate deploy
Prisma schema loaded from prisma\schema.prisma Datasource "db": PostgreSQL database "studyflow", schema "public" at "localhost:5432" 1 migration found in prisma/migrations No pending migrations to apply.
```

SVKM

SHRI VILE PARLE KELAVANI MANDAL'S

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Empowered Autonomous College Affiliated to the University of Mumbai)

DJSCE

Automated Database Seeding & Schema Verification

```
PS C:\Users\D\.gemini\antigravity\scratch\studyflow> npm run db:reset

> studyflow@1.0.0 db:reset > tsx prisma/reset.ts ================================================= 🔄 StudyFlow Database Reset & Seeding CLI Pipeline ================================================= 1. Synchronizing Prisma schema to PostgreSQL... Datasource "db": PostgreSQL database "studyflow", schema "public" at "localhost:5432" The database is already in sync with the Prisma schema. 2. Generating fresh Prisma Client bindings... ✓ Generated Prisma Client (v5.22.0) to .\node_modules\@prisma\client in 114ms 3. Executing automated Faker seed pipeline... 🌱 Starting realistic relational seeding for StudyFlow... 🧹 Cleaned existing database records. 👤 Created 3 core role users: - ADMIN: admin@studyflow.edu (Password: Admin@123) - MEMBER: member@studyflow.edu (Password: Student@123) - GUEST: guest@studyflow.edu (Password: Guest@123) 📚 Populated 4 academic courses for primary member. ✅ Seeded 7 contextual study tasks. 💳 Seeded financial & credit transactions. 📬 Seeded email delivery and bounce telemetry logs. ✨ Seeding completed successfully with full relational integrity! ================================================= 📊 PostgreSQL Database Verification Summary ================================================= - Users: 7 (1 Admin, 5 Members, 1 Guest) - Courses: 4 Academic Modules - Study Tasks: 7 Relational Deliverables - Academic Goals: 3 Term Benchmarks - Study Sessions: 6 Focus Intervals - Transactions: 4 Credit Allocations - Audit Logs: 4 Forensic Records - Email Delivery Logs: 3 Transactional Logs ================================================= ✅ Database reset & seeding pipeline completed successfully!
```

```
Object Explorer ⟶ PostgreSQL 18 ⟶ Databases (studyflow) ⟶ Schemas (public) ⟶ Tables (10)
 ├── User (id, name, email, passwordHash, role, createdAt)
 ├── Session (id, userId, token, expiresAt)
 ├── Account (id, userId, providerId, accountId)
 ├── Course (id, code, title, instructor, credits, color, userId)
 ├── StudyTask (id, title, priority, status, dueDate, courseId, userId)
 ├── AcademicGoal (id, title, progressPercentage, status, userId)
 ├── StudySession (id, title, durationMinutes, focusScore, userId)
 ├── Transaction (id, type, amount, status, referenceCode, userId)
 ├── AuditLog (id, action, entityType, entityId, details, ipAddress, userId)
 └── EmailDeliveryLog (id, recipientEmail, templateName, status, resendMessageId)
```

SVKM

SHRI VILE PARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous College Affiliated to the University of Mumbai)

DJSCE

Persisted Relational Records & Audit Snapshots

```
SELECT id, name, email, role, "createdAt" FROM "User";
```

```
┌─────────┬──────────────────────────┬──────────────────────────┬──────────────────────────┬──────────────┐
(index) │ id │ name │ email │ role │
├─────────┼──────────────────────────┼──────────────────────────┼──────────────────────────┼──────────────┤ │ 0 │
'cmuoj82a200006jk...' │ 'Dr. Eleanor Vance' │ 'admin@studyflow.edu' │ 'ADMIN' │ │ 1 │
'cmuoj82ai00016jk...' │ 'Dhruv Gudka' │ 'member@studyflow.edu' │ 'MEMBER' │ │ 2 │ 'cmuoj82aj00026jk...' │
'Prospective Student' │ 'guest@studyflow.edu' │ 'GUEST' │ │ 3 │ 'cmuoj82am00036jk...' │ 'Leah Hessel' │
'adan_raynor5@gmail.com' │ 'MEMBER' │ │ 4 │ 'cmuoj82ao00046jk...' │ 'Molly Reichert' │
'kenya88@hotmail.com' │ 'MEMBER' │
└─────────┴──────────────────────────┴──────────────────────────┴──────────────────────────┴──────────────┘
```

```
SELECT id, title, course, priority, status FROM "StudyTask";
```

```
┌─────────┬──────────────────────────┬──────────────────────────────────────────┬──────────────┬──────────────┐
│ (index) │ id │ title │ course │ priority │ status │
├─────────┼──────────────────────────┼──────────────────────────────────────────┼──────────────┼──────────────┤
│ 0 │ 'cmuoj82av000...' │ 'Implement Next.js App Router H...' │ 'DJS23AMD302L' │ 'URGENT' │ 'IN_PROGRESS'
│ │ 1 │ 'cmuoj82ax000...' │ 'Configure Zustand Client Store...' │ 'DJS23AMD302L' │ 'HIGH' │ 'COMPLETED' │
│ 2 │ 'cmuoj82ax000...' │ 'Implement Type-Safe Server Act...' │ 'DJS23AMD302L' │ 'HIGH' │ 'TODO' │ │ 3 │
'cmuoj82az000...' │ 'Solve Red-Black Tree Rotation ...' │ 'DJS23AMD301T' │ 'MEDIUM' │ 'COMPLETED' │ │ 4 │
'cmuoj82b0000...' │ 'Implement Dijkstra & A* Pathfi...' │ 'DJS23AMD301T' │ 'HIGH' │ 'IN_PROGRESS' │
└─────────┴──────────────────────────┴──────────────────────────────────────────┴──────────────┴──────────────┘
```

```
SELECT id, recipient, template, status, "resendId" FROM "EmailDeliveryLog";
```

```
┌─────────┬──────────────────────────┬──────────────────────────┬──────────────────────────┬──────────────┐
│ (index) │ id │ recipient │ template │ status │ resendId │
├─────────┼──────────────────────────┼──────────────────────────┼──────────────────────────┼──────────────┤
│ 0 │ 'cmuoj82bj001...' │ 'member@studyflow.edu' │ 'WelcomeStudentEmail' │ 'DELIVERED' │
'msg_ONNDGrmKSANtb9R4' │ │ 1 │ 'cmuoj82bj001...' │ 'member@studyflow.edu' │ 'TaskCreatedEmail' │
'DELIVERED' │ 'msg_VXn55tmhHkFYMuGq' │ │ 2 │ 'cmuoj82bj001...' │ 'bounced.test@example.com' │
'SecurityAlertEmail' │ 'BOUNCED' │ 'msg_W7RWX7E5tPqxNmxE' │
└─────────┴──────────────────────────┴──────────────────────────┴──────────────────────────┴──────────────┘
```

These records demonstrate the complete end-to-end flow from schema creation and automated data seeding to relational persistence, transactional email dispatch, and webhook event logging.

Conclusion

The StudyFlow architecture integrates Next.js application logic with Prisma and PostgreSQL for relational data management, Better Auth for authentication and role-based authorization, Mongoose and MongoDB for unstructured focus telemetry, and React Email with Resend for transactional notifications. The combination of protected backend endpoints, normalized relational entities, automated seed data, transactional database operations, and webhook-based email event persistence provides a complete, robust, full-stack implementation.